

Purely functional state

In this chapter, we'll see how to write purely functional programs that manipulate state, using the simple domain of *random number generation* as the example. Although by itself it's not the most compelling use case for the techniques in this chapter, the simplicity of random number generation makes it a good first example. We'll see more compelling use cases in parts 3 and 4 of the book, especially part 4, where we'll say a lot more about dealing with state and effects. The goal here is to give you the basic pattern for how to make *any* stateful API purely functional. As you start writing your own functional APIs, you'll likely run into many of the same questions that we'll explore here.

6.1 Generating random numbers using side effects

If you need to generate random¹ numbers in Scala, there's a class in the standard library, `scala.util.Random`,² with a pretty typical imperative API that relies on side effects. Here's an example of its use.

Listing 6.1 Using `scala.util.Random` to generate random numbers

```
scala> val rng = new scala.util.Random

scala> rng.nextDouble
res1: Double = 0.9867076608154569

scala> rng.nextDouble
res2: Double = 0.8455696498024141

scala> rng.nextInt
```

Creates a new random number generator seeded with the current system time

¹ Actually, pseudo-random, but we'll ignore this distinction.

² Scala API link: <http://mng.bz/3DP7>.

```
res3: Int = -623297295
```

```
scala> rng.nextInt(10)  
res4: Int = 4
```

← Gets a random integer
between 0 and 9

Even if we didn't know anything about what happens inside `scala.util.Random`, we can assume that the object `rng` has some internal state that gets updated after each invocation, since we'd otherwise get the same value each time we called `nextInt` or `nextDouble`. Because the state updates are performed as a side effect, these methods aren't referentially transparent. And as we know, this implies that they aren't as testable, composable, modular, and easily parallelized as they could be.

Let's just take testability as an example. If we want to write a method that makes use of randomness, we need tests to be reproducible. Let's say we had the following side-effecting method, intended to simulate the rolling of a single six-sided die, which *should* return a value between 1 and 6, inclusive:

```
def rollDie: Int = {  
  val rng = new scala.util.Random  
  rng.nextInt(6)  
}
```

← Returns a random
number from 0 to 5

This method has an off-by-one error. Whereas it's supposed to return a value between 1 and 6, it actually returns a value from 0 to 5. But even though it doesn't work properly, five out of six times a test of this method will meet the specification! And if a test did fail, it would be ideal if we could reliably reproduce the failure.

Note that what's important here is not this specific example, but the general idea. In this case, the bug is obvious and easy to reproduce. But we can easily imagine a situation where the method is much more complicated and the bug far more subtle. The more complex the program and the subtler the bug, the more important it is to be able to reproduce bugs in a reliable way.

One suggestion might be to pass in the random number generator. That way, when we want to reproduce a failed test, we can pass the same generator that caused the test to fail:

```
def rollDie(rng: scala.util.Random): Int = rng.nextInt(6)
```

But there's a problem with this solution. The "same" generator has to be both created with the same seed, and also be in the same state, which means that its methods have been called a certain number of times since it was created. That will be really difficult to guarantee, because every time we call `nextInt`, for example, the previous state of the random number generator is destroyed. Do we now need a separate mechanism to keep track of how many times we've called the methods on `Random`?

No! The answer to all of this, of course, is that we should eschew side effects on principle!